

A Requirements-as-Code Prototype for Explainable Multi-Source Destabilization Monitoring

****Authoring basis:**** Repository-derived project documentation on branch hermes/vmodel-assessment

****Project:**** Country Destabilization Prototype

****Status perspective:**** V1/V2.3 baseline with V4.3.2 event-alignment expansion

Abstract

This paper presents an explainable, requirements-driven prototype for short-term destabilization monitoring across a configurable country set. The system combines a legacy cluster path for socio-political tension, violent escalation, and structural vulnerability with an additive multi-source fusion path spanning event, narrative, governance, market/food, shock, displacement, and structural layers. The approach emphasizes transparent heuristics, explicit uncertainty handling, and machine-readable governance rather than opaque end-to-end prediction. Engineering-wise, the project is implemented as a modular Python system with a FastAPI MVP and a V-Model-Light / requirements-as-code baseline. At the current documented state, the repository contains 26 stakeholder requirements, 43 system requirements, 124 software requirements, 156 governed verification artifacts, and 2462 function-level trace links. We argue that the main contribution of the project is not a final predictive model, but a reproducible and extensible architecture for traceable multi-source analytical monitoring. The current limitations lie in calibration, event/data governance, semantic normalization, and productization of dedicated API/frontend read models.

1. Introduction

Monitoring politically and socially destabilizing developments is challenging because heterogeneous signals evolve at different temporal scales and with different semantic reliability. Protest-related media signals, conflict events, structural fragility, governance stress, displacement, shocks, and narrative dynamics rarely align perfectly in time or meaning. A practical monitoring system therefore faces two coupled problems: first, how to transform heterogeneous evidence into coherent analytical signals; second, how to do so in a way that remains inspectable, testable, and reviewable.

The Country Destabilization Prototype addresses this as an engineering-first analytical prototype. The system is designed to expose short-term anomalies and plausible destabilization indicators, not to replace human judgment. Its central design choice is to treat explainability, traceability, and validation as first-class concerns. This is reflected in the project architecture, in the explicit separation between score and confidence, and in the V-Model-Light governance layer that links requirements, methods, code, and verification artifacts.

2. System Context and Scope

The prototype currently operates on a default 10-country baseline: Germany, Israel, Iran, Ukraine, Russia, Japan, China, Taiwan, Poland, and Nigeria. The repository documents a historical evolution from a narrower three-country origin toward a configurable baseline. The operational scope includes:

- legacy event/context sources: GDELT, UCDP, bridge events, country context,
- additive V3+ sources: FAO FFPI, FAO FPMA, UN Comtrade, GDACS, UNHCR, narrative input, governance input, and GDELT event observations,

- output spaces: snapshot, historical rolling trend, monthly historical fusion, validation artifacts, plots, handout, run metadata, run comparisons, and review bundles.

The project explicitly excludes fully autonomous decision-making, complete real-time global coverage, black-box fusion, and final scientific claims of causal prediction.

3. Related Work Framing

The prototype is positioned at the intersection of several established research and engineering traditions.

First, it aligns with early-warning and instability-monitoring systems that combine heterogeneous risk indicators rather than relying on a single event stream. Typical systems in this space integrate political, social, economic, or conflict-related indicators to provide structured analyst support.

Second, it is related to model-based risk scoring and indicator aggregation approaches. However, unlike many score-only systems, this prototype explicitly maintains separate score and confidence paths and surfaces freshness, dominance, and coverage diagnostics for analyst review.

Third, the project shares concerns with explainable AI and interpretable analytics. Its methods are deliberately heuristic and transparent: weighted means, threshold mappings, rolling windows, and reviewable alignment heuristics are preferred over difficult-to-audit end-to-end black-box models.

Fourth, from an engineering perspective, the project belongs to the broader class of requirements-driven and traceability-centric software systems. The use of machine-readable requirements, governed verification artifacts, and function-level trace links makes the prototype relevant not only as an analytical system but also as a case study in disciplined analytical software engineering.

This document does not claim a full external literature review. Instead, it introduces a formal related-work structure that can be extended with domain literature on early warning systems, risk indicator fusion, explainable analytics, and requirements-as-code governance.

4. Architecture

The architecture follows a strict layered separation between raw sources, feature/observation derivation, scoring/fusion logic, reporting, API access, and governance artifacts.

[Abbildung: System architecture -> diagrams/architecture_overview.svg]

At the repository level, this separation is reinforced through dedicated directories for product code, tests, tools, scripts, machine-readable governance artifacts, and generated outputs.

[Abbildung: Repository structure -> diagrams/repository_structure.svg]

The main implementation areas are:

- proto.pipeline: orchestration and configuration loading,
- proto.sources: source adapters,

- proto.features: legacy cluster-feature builders,
- proto.scoring: legacy cluster scoring and rolling history,
- proto.observations: canonical observation schema and normalization,
- proto.fusion: multi-source fusion, confidence, freshness, and validation,
- proto.reporting: exports, plots, handout generation,
- proto.runs: run metadata, run comparison, reference handling,
- api: FastAPI MVP for health, options, runs, status, and artifacts.

5. Methods

5.1 Legacy cluster path

The legacy path computes features on a 0 to 100 scale and aggregates them into subscores and cluster scores. For a country c , cluster k , and time t , the cluster score is defined as:

$$C_{\{c,k,t\}} = \frac{1}{n_k} \sum_{j=1}^{n_k} S_{\{c,k,j,t\}}.$$

The current threshold configuration maps scores to four stages using 30, 56, and 80 as boundaries. Trends are derived from robust 7-day vs. baseline-window comparisons with $\delta_{\epsilon} = 1.25$.

5.2 Historical rolling logic

The historical path uses a rolling-window approach with $\text{window_days} = 7$, $\text{min_valid_days} = 5$, and $\text{aggregation} = \text{rolling_mean}$. Importantly, the snapshot is derived from the same historical logic rather than from an independent scoring path. This yields methodological coherence between current-state outputs and historical trajectories.

5.3 Canonical observations and fusion

New sources are normalized into a common observation schema containing source identity, temporal range, layer/group context, raw value, normalized value, provenance, and quality completeness. Source-to-group mappings are explicit.

[Abbildung: Source-to-layer mapping -> [diagrams/source_to_layer_mapping.svg](#)]

A group score is computed as a weighted average of normalized source signals, adjusted by freshness or decay where applicable:

$$G_{\{c,g,t\}} = 100 \cdot \frac{\sum_i w_i s_{\{i,c,g,t\}} d_{\{i,c,g,t\}}}{\sum_i w_i}.$$

The fusion total score is then computed across available groups:

$$F_{\{c,t\}} = \frac{\sum_g W_g G_{\{c,g,t\}}}{\sum_g W_g}.$$

5.4 Confidence and validation

A core design principle is that score and confidence remain separate. Group confidence is derived from source coverage, recency, completeness, and consistency. Total confidence additionally includes penalties for excessive single-group dominance and a high share of limited groups. This prevents the system from presenting strong scores without simultaneously exposing evidential weakness.

Validation focuses on analytical plausibility rather than truth certification. The current validation layer includes:

- ranking plausibility,
- reference-episode checks,
- freshness diagnostics,
- historical responsiveness profiles,
- yearly country profiles,
- peak detection,
- peak attribution,
- event alignment against an event registry,
- review queues for weak or ambiguous event grounding.

5.5 Score semantics and interpretation boundaries

The system's scores must be interpreted as modeled analytical states rather than direct real-world outcome probabilities. A high legacy cluster score indicates strong aggregated signal intensity in the configured feature families. A high fusion score indicates weighted anomaly intensity across multiple layers under the configured freshness, coverage, and decay logic. Neither should be read as a direct probability of regime breakdown, war, or other end-state outcomes.

This distinction matters especially when score and confidence diverge. High score with low confidence indicates strong apparent anomaly under weak or aging evidence. Low score with high confidence indicates relatively robust non-anomaly in the observed model. The architecture is deliberately designed to preserve this distinction instead of collapsing it into a single opaque number.

5.6 Method boundaries

The prototype is intended for short-term signal detection and analyst support. It is not a complete geopolitical causal model, not a long-range predictive engine, and not a substitute for domain judgment. Layer-specific limitations also remain important: event layers depend on registry and coverage quality, narrative layers remain sensitive to semantic taxonomy quality, governance and structural layers are informative but comparatively slow-moving, and auxiliary layers such as shock or displacement inherit external periodicity and completeness constraints.

The methodological bottom line is therefore: score represents model state under explicit assumptions, while event alignment represents grounding support rather than proof of causality.

6. Outputs and Workflow

A standard run loads the configuration and sources, computes legacy features and cluster scores, transforms additive sources into canonical observations, calculates fusion signals, writes historical and

validation outputs, and finally produces plots, handout, metadata, and run comparisons.

[Abbildung: Program flow -> diagrams/program_flow.svg]

The main generated outputs are written under `outputs/runs/current_run/` and include CSV/JSON exports, handout markdown, plots, run metadata, run comparison, and reference comparison.

7. Governance and Verification

The project implements a machine-readable V-Model-Light target structure in `vmodel/`. This currently contains:

- 26 stakeholder requirements,
- 43 system requirements,
- 124 software requirements,
- 156 implemented verification artifacts,
- 46 governed open issues,
- 2462 function-level trace links.

[Abbildung: V-model traceability -> diagrams/vmodel_traceability.svg]

This governance structure is one of the key technical contributions of the prototype. It transforms what could otherwise be a loosely evolving analytical codebase into a reviewable system where functions, tests, algorithms, and requirements can be linked explicitly.

8. Current State Assessment

The current state is strong for a prototype along three dimensions:

1. Architecture: modular, layered, and extensible.
2. Method transparency: clear heuristics, explicit configuration, separated confidence, and visible validation artifacts.
3. Governance: unusually strong traceability and verification discipline for an analytical prototype.

At the same time, the system remains a prototype rather than a finalized scientific or operational product. Calibration remains incomplete, event and semantic governance require further hardening, and the FastAPI layer still reflects MVP scope rather than full product-grade read models.

9. Limitations

The most relevant current limitations are:

- heuristic rather than empirically finalized weighting and thresholding,
- partial dependence on curated event registries and structured inputs,
- limited semantic governance for narrative inputs,
- incomplete productization of specialized API endpoints,
- no claim of final predictive validity,
- related-work framing present, but external literature integration still to be completed.

10. Evaluation Outlook

A disciplined next step is an explicit evaluation program. At minimum, this should test four questions:

1. Signal quality: do cluster and fusion outputs identify known episodes at plausible times?
2. Calibration robustness: how sensitive are rankings and peaks to weights, thresholds, and decay parameters?
3. Uncertainty quality: do confidence values and penalties correlate with better or worse grounding quality?
4. Analyst utility: do review queues, dominance diagnostics, and handout artifacts improve real review workflows?

This suggests a concrete evidence package for future work: curated event/reference sets, annotated multi-country case studies, parameter sweeps, and expert-review protocols.

11. Conclusion

The Country Destabilization Prototype demonstrates a useful pattern for explainable multi-source monitoring under strong software-governance constraints. Its primary value lies in combining analyst-oriented signal generation with requirements-as-code traceability and validation infrastructure. This makes it a compelling foundation for future methodological calibration, API/frontend productization, and a more formal scientific evaluation program.

References

Current repository-grounded references:

1. README.md
2. IMPLEMENT.md
3. docs/requirements/ProtoScope.md
4. docs/requirements/ProtoSR.md
5. docs/requirements/ProtoSyR.md
6. docs/requirements/ProtoSwR.md
7. docs/method/ProtoMethod.md
8. docs/method/ProtoScoring.md
9. docs/method/ProtoRunbook.md
10. docs/testing/TestStrategy.md
11. docs/traceability/TraceabilityMatrix.md
12. vmodel/requirements/*.yaml
13. vmodel/verification/test_specifications.yaml
14. vmodel/change/open_issues.yaml
15. vmodel/traceability/trace_links.yaml

Automatisch erzeugte PDF-Exportfassung. SVG-Abbildungen werden in dieser Minimalfassung als referenzierte Diagrammhinweise geführt.